异常处理

程序中常见的错误有两大类

这类错误比较隐蔽，不易被发现，往往耗费许多时间和精力。这成为程序调试中的一个

类声明最后没有；

函数变量名拼写错误；

缺}

语法错误

编译时，编译系统能发现程序中的语法错误

运行错误

数组越界；

除0；

压栈时满栈、出栈空栈；

文件打开失败；

能通过编译，但得不到正确的运行结果，甚至导致程序不正常终止，或出现死机现象

异常处理

在设计程序时，应当事先分析程序运行时可能出现的各种意外的情况，并且分别制订出相应的处理方法，这就是程序的异常处理的任务。

- 在运行没有异常处理的程序时，如果运行情况出现异常，由于程序本身不能处理，程序只能终止运行。
- 如果在程序中设置了异常处理机制，则在运行情况出现异常时，由于程序本身已规定了处理的方法，于是程序的流程就转到异常处理代码段处理。用户可以指定进行任何的处理。

异常处理

面向对象的 C++程序设计语言可以编写大型和十分复杂的程序，往往会产生一些很难查找的甚至是无法避免的运行时错误。

大型程序各部分可能是由不同的小组编写的，它们由公共接口连接，错误可能就发生在相互的配合上，也可能发生在事先根本想不到的个别的条件组合上。

异常处理在 C++编程中已经普遍采用，成为提高程序稳健性的重要手段之 一。

C++异常处理机制

C++处理异常的机制是由3个部分组成的，检查(try)、抛出(throw)和捕捉(catch)：

◆try块中放需要检查的语句；

throw抛出什么样的数据由程序设计者自定，可以是任何类型的数据。

◆throw用来当出现异常时发出一个异常信息；

throw 表达式;

表达式结果的数据类型被用于区分不同类型的异常

◆catch用来捕捉异常信息，如捕捉到异常，处理它

例1：给出三角形的三边a,b,c，求三角形的面积

```
#include <iostream>
#include <cmath>
using namespace std;
int main()
{
        double triangle(double,double,double);
        double a,b,c;
        cin>>a>>b>>c;
        try
        {
                while(a>0 && b>0 && c>0)
                 {cout<<triangle(a,b,c)<<endl;
                 cin>>a>>b>>c;}
         }
```

◆只有a+b>c,b+c>a,c+a>b时，才能构成三角形。设置异常处理，对不符合三角形条件的输出警告信息，不予计算。

```
catch(double) {
cout<<"a="<<a<<",b="<<b<<",c="<<c<<",that is
not a triangle!"<<endl;
}
 cout<<"end"<<endl;
 return 0;
}
```

用catch捕捉异常信息并作相应处理

代码

```
double triangle(double a,double b,double c)
{
    double s=(a+b+c)/2;
    if (a+b<=c||b+c<=a||c+a<=b) throw a;
    return sqrt(s*(s-a)*(s-b)*(s-c));
}
```

当不符合三角形条件抛出异常信息

程序进行异常处理过程

首先把可能出现异常的、需要检查的语句或程序段放在try后面的花括号中。

程序开始运行后，按正常的顺序执行到try块，开始执行try块中花括号内的语句：

情况1：如果执行try块内语句过程中没有发生异常，则catch子句不起作用，流程转到catch子句后面的语句继续执行。

情况2：如果在执行try块内的语句（包括其所调用的函数）过程中发生异常，则throw运算符抛出一个异常信息。throw抛出异常信息后，流程立即离开本函数或块

- ✓ 异常信息提供给try-catch结构，系统会寻找与之匹配的catch子句。
- ✓ 在进行异常处理后，程序并不会自动终止，继续执行catch子句后面的语句。

例2：除0异常处理

```
#include <iostream>
 using namespace std;
int main()
 {
        cout << “输入两个整数： ";
        int number1, number2;
        cin >> number1 >> number2;
        try {
              if (number2 == 0)  throw number1;
            cout << (number1 / number2) << endl;
         }
        catch (int) {
            cout << “ 不能除0" << endl;
         }
        cout << "Execution continues ..." << endl;
        return 0;
}
```

try-catch块

可能出现异常的、需要检查的语句

抛出异常

捕获异常处理

异常处理完程序继续执行

说明:

- ☆ 被检测的代码必须放在try块中，否则不起作用。
- ☆ try块和catch块作为一个整体出现，catch块是try-catch结构中的一部分，必须跟在try块之后，不能单独使用，在二者之间也不能插入其他语句。
- ☆ try和catch块中必须有用花括号括起来的复合语句，即使花括号内只有一个语句，也不能省略花括号。

例3：一个try块抛出多个异常

```
#include <iostream>
using namespace std;
int main()
{
    cout << “输入10~40的整数: ";
    int number;
    cin >> nubmer;
    try
    {
      if (number >40 || number <10)
            throw number;
       cout << "End of try block ..." << endl;
    }
   catch (int){cout << "The number is  illegal" << endl;}

   cout << "Continue ...  << endl;
   return 0;
}
```

- ✓ 一个try-catch结构中只能有一个try块，但却可以有多个catch块，以便与不同的异常信息匹配。
- ✓ 如果需要检测多个不同的异常信息，应当由throw抛出不同类型的异常信息

catch只检查所捕获异常信息的类型，而不检查它们的值，一般只写异常信息的类型名

☆ 有时希望在捕获异常信息时，还能利用throw抛出的值，catch除了指定类型名外，也可指定变量名，如：

```
catch (int num)  {cout <<"The number "<<num<< "is too big" << endl;}
```

若抛出的异常信息是int型变量number，则catch在捕获异常信息number时，还使num获得number的值。

☆ 若catch子句中没有指定异常信息的类型，用删节号“...”，则表示它可以捕捉任何类型的异常信息，如

```
catch(...)   {cout<<"OK"<<endl;}
```

注：这种catch子句应放在try catch结构中的最后，相当于“其他”。如果把它作为第一个catch子句，则后面的catch子句都不起作用。

例4：压栈和出栈

栈满和栈空时原来的处理

```
template <typename T>void
Stack<T>::Push(const T &data)
{
    assert(!IsFull());
    elements[++top]=data;
 }
template<typename T>T Stack<T>::Pop()
{
    assert(!IsEmpty());
    elements[top--];
}
```

压栈，栈满则退出

出栈，栈空则退出

例4：压栈和出栈

✓ 异常信息可以是C++系统预定义标准类型，也可是用户自定义的类型

```
template <typename T>void
Stack<T>::Push(const T &data)
{
   if(IsFull())
   throw pushOnFull<T>(data);
    elements[++top]=data;
 }
template<typename T>T Stack<T>::Pop()
{
      if(IsEmpty())
      throw popOnEmpty<T>();
      elements[top--];
}
```

类对象

类对象

异常处理：

栈满抛出一个异常

栈空抛出一个异常

```
template<typename T>class pushOnFull
{
        T value;
    public:
        pushOnFull(T i) { value=i;}
        T value() { return  value;}
        void print()
         {cerr<<"栈满，"<<value()<<"未压入栈"<<endl; }
};
```

栈满异常类

栈空异常类

```
template<typename T>class popOnEmpty
{
   public:
        void print(){cerr<<"栈已空，无法出栈"<<endl;}
};
```

```
int main()
{
    int a[9]={1,2,3,4,5,6,7,8,9},b[9]={0},i;
    Stack<int>istack(8);
    try{
      for(i=0;i<9;i++) istack.Push(a[i]);
      istack.PrintStack();
      }
    catch(pushOnFull<int>&eObj){eObj.print();}
    try{
      for(i=0;i<9;i++) b[i]=istack.Pop();
       }
    catch(popOnEmpty<int>&eObj){eObj.print();}
    for(i=0;i<8;i++) cout<<b[i]<<'\t';
    cout<<endl;
    return 0;
}
```

压栈

出栈

```
int main( )
{
   cout<<"Test\n";
  try{
        cout<<"Try\n";
        cout<<fun(8)<<endl;
        cout<< fun (16)<<endl;
        cout<< fun (21)<<endl;
        cout<< fun (15)<<endl;
        cout<< fun (4)<<endl;
   }
   catch(int )       {  cout<<" A multiplier of 3.\n ";    }
   catch(char){ cout<<"A multiplier of 5.\n";  }
   catch(double){ cout<<  “A multiplier of 7.\n” ; }
   cout<<"The end.\n";
   return 0;
}
```

```
int fun(int x)
{
  double d=2.3;
 if(x%3==0) throw x;
else if(x%5==0)
        throw 'a';
else if(x%7==0)
        throw d;
return x;
}
```

栈展开

因发生异常而逐步退出复合语句和函数定义，被称为**栈展开**。

随着栈展开，在退出的复合语句和函数定义中定义的局部变量的生命期也结束了，在栈中分配的局部量占用的资源也被释放，由系统回收。

栈展开与异常捕获过程

如果throw表达式位于try块中，则检查与try块相关联catch子句列表，看是否有子句能够处理该异常

如果有匹配的，则该异常被处理

若找不到匹配catch子句，则退到主调函数中继续查找

如果退到主调函数时有一个被抛出的异常未能处理，且这个调用位于一个try块中，检查与该try块相关联catch子句列表有子句匹配

如果有，处理该异常

这个查找过程逆着嵌套的函数调用链向上继续，直到找到处理该异常的catch子句。

如果没有，则查找过程进一步在该函数的主调函数中继续进行

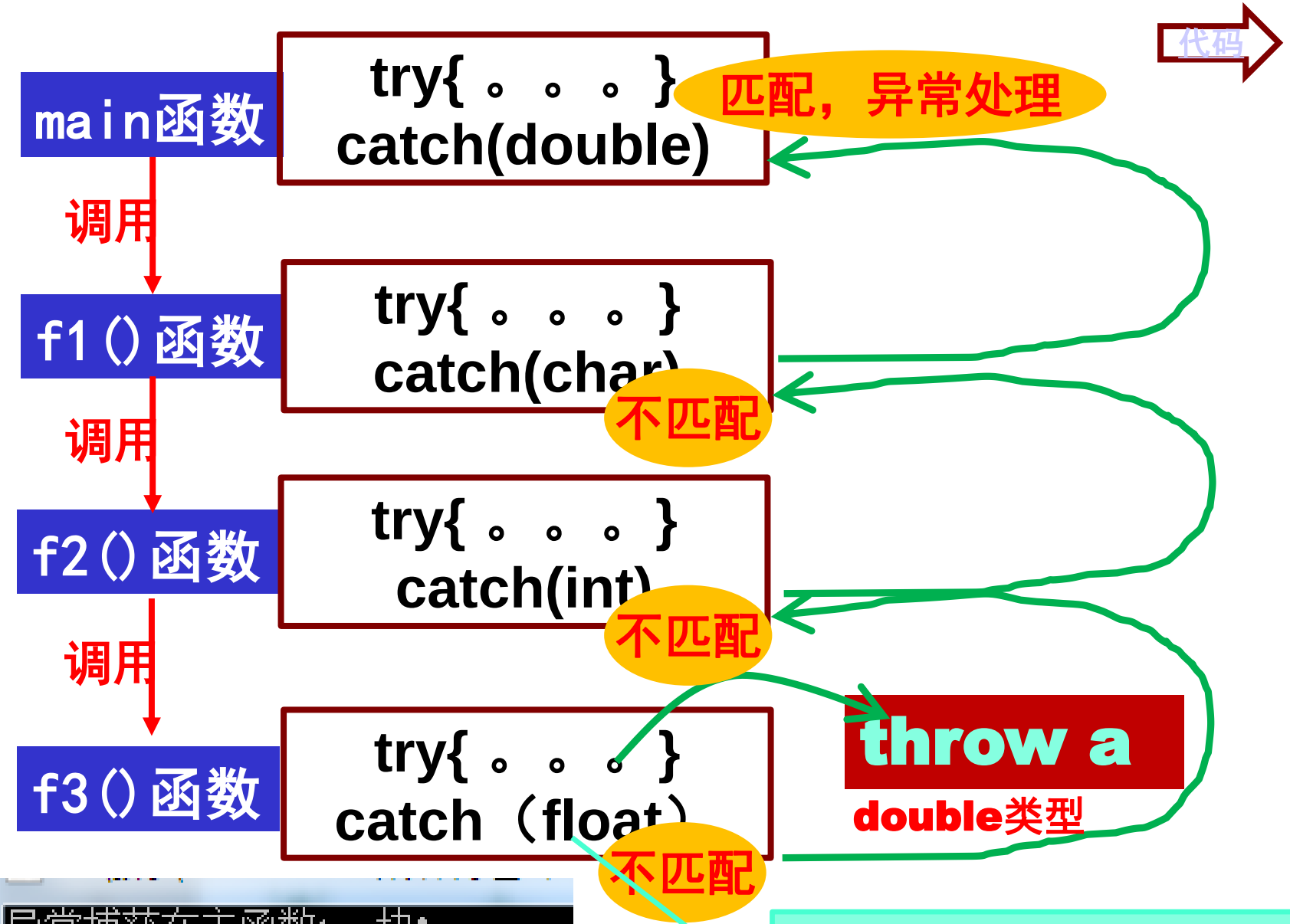
如果最终没有找到匹配的catch子句，程序就调用C++标准库中定义的函数terminate()

```
int main( )
{
        try  { f1(); }
        catch(double)
        { cout<<"异常捕获在主函数try块!"<<endl; }
        cout<<"main 函数结束！ "<<endl;
        return 0;
}
```

```
void f1()
{       try   {f2();}
        catch(char) {cout<<"异常捕获在f1 try 块中！ ";}
        cout<<"函数f1()结束！ "<<endl;
}
```

```
void f2()
{       try   {f3();}
        catch(int) {cout<<"异常捕获在f2 try 块中！ ";}
        cout<<"函数f2()结束！ "<<endl;
}
```

```
void f3()
{       double a=0;
        try   {throw a;}
        catch(float) {cout<<"异常捕获在f3 try 块中！ ";}
        cout<<"函数f3()结束！ "<<endl;
}
```


代码
main函数
try{。。。}
catch(double)
匹配，异常处理
调用
f1()函数
try{。。。}
catch(char)
不匹配
调用
f2()函数
try{。。。}
catch(int)
不匹配
调用
f3()函数
try{。。。}
catch（float）
不匹配
throw a
double类型
异常捕获在主函数try块!
main 函数结束!
Press any key to continue
如果改为double，程序运行结果？

► 栈展开期间，当一个复合语句（或语句块）或函数退出时，在退出的域中有某个局部量是类对象，栈展开过程将自动调用该对象的析构函数，完成资源的释放。

```
#include <iostream>
#include <string>
using namespace std;
class Student{
public:
   Student(int n,string nam)  {
        cout<<"constructor-"<<n<<endl;
        num=n;name=nam;}
~Student (){cout<<"destructor-"<<num<<endl;}
   void getdata();
private:
int num;
string name;
 };
```

```
void Student::getdata()
{
        if(num==0) throw num;
        else cout<<num<<" "<<name<<endl;
        cout<<"in getdata()"<<endl;
 }
void fun()
{
        Student stud1(1101,"Tan");
        stud1.getdata();
        Student stud2(0,"Li");
        stud2.getdata();
}
```

```
int main()
{
        cout<<"main begin"<<endl;
        cout<<"call fun()"<<endl;
        try  {fun();}
        catch(int n)
        {cout<<"num="<<n<<",error!"<<endl;}
        cout<<"main end"<<endl;
        return 0;
}
```

- 若函数动态获得过资源(如用new运算符取得的资源和打开的文件），因异常栈展开时这些资源的释放语句可能被忽略，导致这些资源将永远不会被自动释放。

☆ 最好把资源封装在一个类中，由类的析构函数自动释放资源

采用面向对象的程序设计， 取得资源的动作封装在类的构造函数中，释放资源的动作封装在类的析构函数中。当一个函数带着未处理的异常退出时，函数中这种类对象被自动销毁，资源 （包括动态空间分配的资源和打开的文件）释放。

栈展开过程绝不会跳过封装在 类的析构函数中的资源释放动作。所以由文件重构对象应该放在构造函数中，而把对象存入文件应该放在析构函数中

异常重新抛出

当 catch 语句捕获一个异常后，可能不能完全处理异常，在完成某些操作后， catch 子句可能决定该异常必须由函数链中更上级的函数来处理，这时 catch 子句可以重新抛出该异常，把异常传递给函数调用链中更上级的另一个 catch 子句，由它进行进一步处理。

重新抛出表达式为：

throw;

仅有一个关键字，异常类型在 catch 语句中已有，不必再指明。被重新抛出的异常就是原来的异常对象

```
void fun1()
{
   int *res;
  res=new int[100];
  try{ /*可能引起异常抛出的操作代码*/  }
  catch(...)
  {
     delete [] res;
     throw;
   }
   delete [] res;
}
```

不论是那种异常
都在此释放

释放资源对象 res

重新抛出异常

正常退出前释放资
源对象 res

异常和继承

当类的层次结构用于异常时，异常处理的方式变得更加多样化，表示异常的类通常被组成为一个组（group）或者一个层次结构

例：对由栈类成员函数抛出的异常

```
class popOnEmpty{...};
class pushOnFull{...};
```

可定义一个Excp的基类，再从该基类派生出这两个异常类

```
class Excp{
  public:
 void print(string msg){cerr<<msg<<endl;}
};
```

```
class popOnEmpty:public Excp{...};
class pushOnFull:public Excp{...};
```

层次结构

```
class Excp{...}; //所有异常类的基类
class stackExcp:public Excp{...}; //栈异常类的基类
class popOnEmpty:public stackExcp{...}; //栈空，退栈异常
class pushOnFull:public stackExcp{...}; //栈满，压栈异常
class mathExcp:public Excp{...}; //数学库异常的基类
class zeroOp:public mathExcp{...}; //数学库零操作异常
class divideByZero:public mathExcp{...}; //数学库被零除异常
```

在处理类类型异常时，**catch**子句的排列顺序非常重要。当异常被组织成类层次结构时，类类型的异常可被该类类型的所有基类的**catch**子句捕获到。如**pushOnFull**类类型的异常可以由**stackExcp**或**Excp**类类型异常所对应的**catch**子句处理.

异常规范

为了增强程序的可读性和可维护性，使程序员在使用一个函数时就能看出这个函数可能会抛出哪些异常，C++ 允许在函数声明和定义时，加上它所能抛出的异常的列表：

```
void func() throw (int, double);

void func() throw ();

void func();
```

表明 func 可能抛出 int 型、double 型的异常

表明func 函数不会抛出任何异常。

若不写能抛出哪些类型的异常，可抛出任何类型的异常

C++标准库的异常类

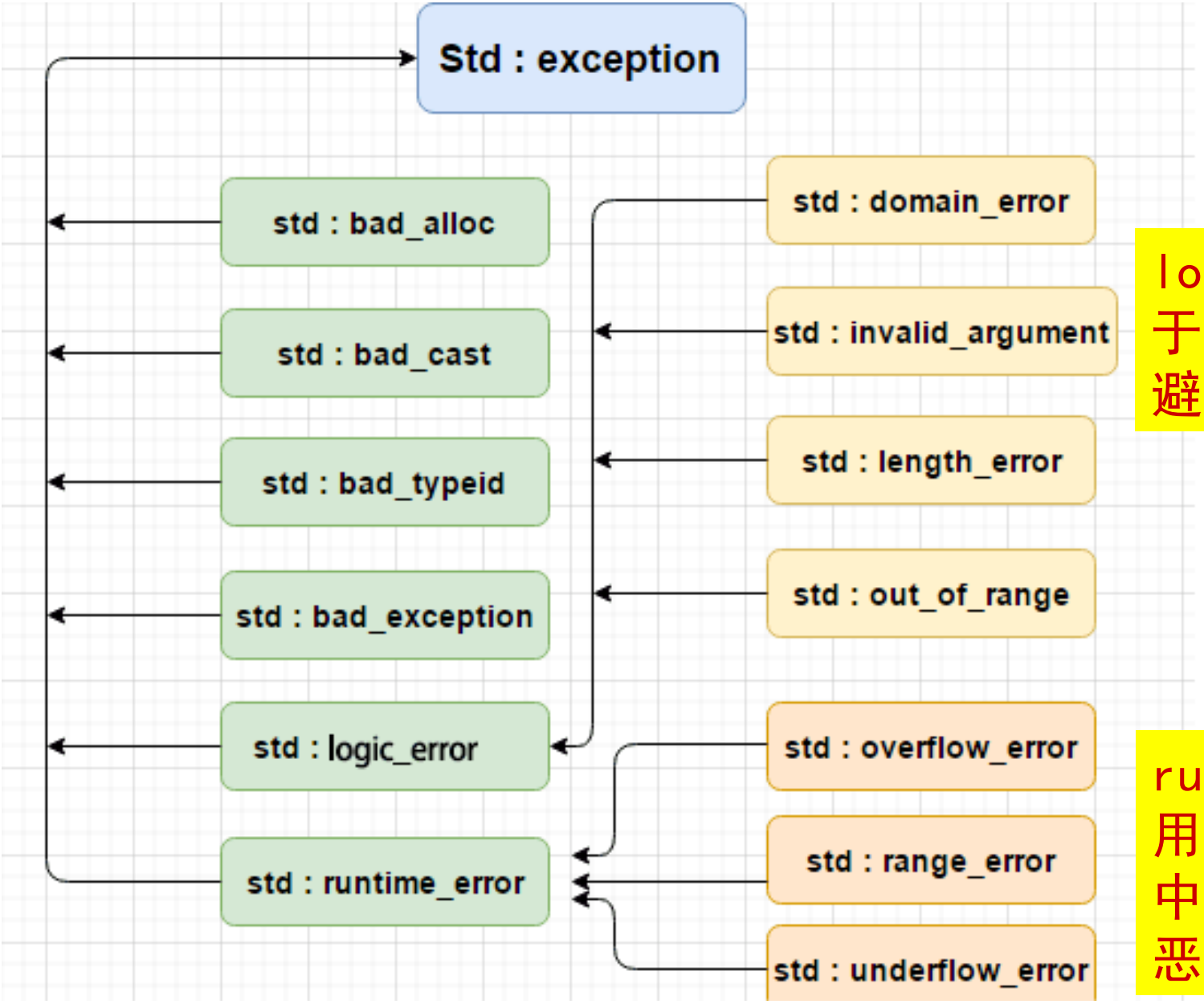
C++标准库提供了一个异常类层次结构，用来报告 C++标准库中的函数执行期间遇到的程序不正常情况。这些异常类也可以被用在用户编写的程序中，或被进一步派生来描述程序中的异常。

C++标准库中的异常层次的根类为 exception，是 C++标准库函数抛出的所有异常类的基类，接口如下：

```
class exception{
public:   exception()  throw();
exception(const exception &)  throw();
exception &operator=(const exception&)
throw();
virtual ~exception()  throw();
virtual const char*what() const  throw();
};
```

返回一个 C 风格的字符串

使用预定义的异常类，程序须包含<stdexcept>



logic_error用于处理程序中可避免逻辑错误，

runtime_error用于处理程序中无法处理的恶性错误。